Haetae

Go 로 만든 독립 RPKI Relying Party

The Haetae is a one-horned guardian of Korean legend that can tell truth from lies.

This one checks whether a route is telling the truth.



2026-11 · v0.2.0 · 시험 운영 중 · 오픈 소스 (ISC)

왜 또 하나의 RP 인가

RPKI 의 결과는 RP 하나에 달려 있습니다

- 라우터는 RP 가 준 VRP 로 경로를 받거나 버립니다. RP 의 버그나 장애는 곧 라우팅 장애입니다
- 서로 다른 구현의 RP 를 둘 이상 두면, 한쪽의 문제를 다른 쪽이 드러냅니다
- 그런데 지금 운영에 쓸 만한 공개 RP 는 많지 않습니다

Haetae 는 기존 RP 와 코드를 나누지 않는 독립 구현입니다

Routinator	Rust · NLnet Labs	운영 중
rpki-client	C · OpenBSD	운영 중
FORT	C · NIC.mx	운영 중
rpki-prover	Haskell · 개인 개발	운영 중
OctoRPKI	Go · Cloudflare	보관됨
RIPE NCC Validator	Java · RIPE NCC	종료

한눈에 보는 Haetae

0

rpki-client 와의 차이

VRP · ASPA · 라우터 키 (같은 데이터로 검증)

42초

전체 검증 시간

같은 캐시에서 rpki-client 는 16 분

0

외부 의존 라이브러리

Go 표준 라이브러리만 사용

1

실행 파일

수집 · 검증 · RTR · API 가 하나로

모든 표준 RPKI 객체 를 검증하고, 라우터에는 **RTR v0 / v1 / v2**, 시스템에는 **HTTP API** 로 줍니다

약 11,400 줄의 Go 코드 · 시험 69 개 (경쟁 상태 검사 · fuzz 포함) · 10 분 주기 상주 운영 중 · 시험 라우터에 RTR 연동

다른 RP 와 비교

2026-11 기준 · 각 프로젝트 공개 문서 기준

	Haetae	Routinator	rpki-client	FORT
만든 곳 · 언어	**Go**	NLnet Labs · Rust	OpenBSD · C	NIC.mx · C
라이선스	**ISC**	BSD-3	ISC	MIT
외부 의존	**없음 (Go 표준만)**	Rust 패키지	OpenSSL 등	OpenSSL 등
RTR 서버	**내장 (v0 / v1 / v2)**	내장	별도 (StayRTR 등)	내장
ASPA	**지원**	지원 (옵션)	지원	시험판 (1.7.0)
BGPsec 라우터 키	**지원**	지원 (옵션)	지원	지원
경로 판정 · 변경분 API	**지원 (ASPA 경로 포함)**	지원	없음	없음
TA constraints	**쓰지 않음 (RFC 기준)**	쓰지 않음	어기면 거부	쓰지 않음
쓰임	**시험 운영**	가장 널리 쓰임	널리 쓰임	운영 중

구조 : 받고, 가리고, 내어 줍니다

1 수집

- RRDP 우선, 실패하면 rsync
- 저장소 동시 32 곳
- 저장소당 5 분 제한

2 검증

- TAL 부터 CA 를 따라 Manifest 까지
- 서명 · 해시 · CRL · 유효기간
- 자식 자원은 부모 안에

3 집합

- VRP · ASPA · 라우터 키
- SLURM 로컬 예외
- serial 과 변경분 (delta)

4 제공

- RTR v0 / v1 / v2
- HTTP API
- JSON · CSV 파일

10 분마다 반복 · 검증에 실패해도 직전 정상 결과를 계속 제공 · 새 판으로 바꿔도 라우터 연결이 끊기지 않음

RFC 6487 / 3779 / 6488 / 9286 / 9582 / 8182 / 8210 / 8416 / 8630

저장소에 게시되는 모든 표준 객체를 검증

RFC 표준 객체는 하나도 빼지 않고, 검증한 객체 약 57 만 개를 모두 조회할 수 있게 남깁니다

TA · CA 인증서
RFC 6487 · 3779

CRL
RFC 6487

Manifest
RFC 9286

ROA → VRP
RFC 9582

ASPA
draft aspa-profile

BGPsec 라우터 키
RFC 8209

GBR 연락처
RFC 6493

SPL
draft rpki-prefixlist

TAK
RFC 9691

표준이 아닌 확장자 (사설 · 시험 객체) 는 개수만 집계

세 RP 의 결과 비교

2026-11-01 매일 비교 · 각 RP 가 따로 수집 · 검증한 VRP 를 하나하나 대조했습니다

Haetae

1,030,627

기준

Routinator

1,030,627

Haetae 에서만 12
Routinator 에서만 12

rpki-client

1,030,613

Haetae 에서만 27
rpki-client 에서만 13

남은 차이는 수집 시각 차이입니다 : 세 RP 가 몇 분 ~ 수십 분 간격으로 따로 수집하므로, 그 사이 바뀐 ROA 가 차이로 셉니다 (전체의 0.004% 이하)

같은 입력이면 차이 0 : 같은 캐시 · 같은 검증 시각으로 rpki-client 와 대조하면 VRP · ASPA · 라우터 키 모두 같습니다 (고정 스냅샷, Haetae 가 직접 받은 캐시 모두)

constraints 파일 갱신 전 (2026-10-31 까지) 에는 rpki-client 가 약 2,800 개 적었습니다 (9 쪽)

빠른 검증, 짧은 주기

같은 서버 · 같은 캐시 (약 57 만 객체) 를 오프라인으로 검증한 시간 · 각각 혼자 실행



2026-10 측정 · rpki-client 9.0 = Ubuntu 24.04 패키지 · Haetae 는 검증을 여러 코어에 나눠 돌림

1~2 분
수집 + 검증 한 회차 (10 분 주기)

2 초
판 올리기 RTR 멈춤 (끊김 없음)

약 5 GB
상주 메모리 (회차 포함)

TA constraints 를 쓰지 않는 이유

rpki-client 는 RIR 별 자원 범위 파일 (.constraints) 로 객체를 거릅니다

- RFC 에 없는 rpki-client 고유 기능입니다. 파일은 rpki-client 개발자가 만들어 함께 배포합니다
- 파일이 IANA 할당보다 늦으면, 새로 할당된 자원의 정상 객체가 거부됩니다
- 실측 : Ubuntu 24.04 패키지 (rpki-client 9.0) 의 파일 (2024-04) 에는 2024-11 APNIC 에 할당된 2410::/12 가 없음

Haetae 는 constraints 를 쓰지 않습니다

RFC 그대로 (Routinator · FORT 와 같음) · RIR 의 TA 는 모든 자원을 가지는 것이 표준 운영

패키지 파일 (2024-04)

2,773 VRP · 59 ASPA

정상 객체 거부

최신 파일 (2026-07) 로 바꾼 뒤

차이 0

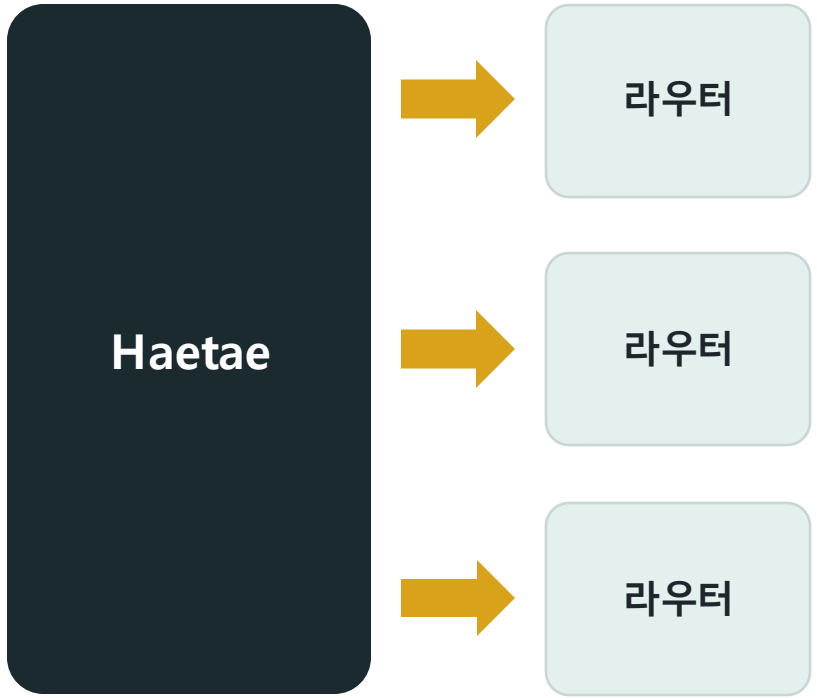
Haetae 와 같은 결과

2026-10 측정 · 같은 데이터를 같은 rpki-client 9.0 으로, constraints 파일만 바꿔 검증 · 차이 = constraints 없이 검증한 결과와 비교

라우터에는 RTR 로

Cisco · Juniper · Nokia · Huawei · FRR 등 RTR 을 지원하는 라우터에 VRP 를 줍니다 (시험 : Cisco IOS XR)

v0	**RFC 6810**	가장 오래된 장비까지
v1	**RFC 8210**	Router Key (BGPsec) 포함
v2	**draft 8210bis**	ASPA 까지 (경로 누출 방지)
Δ	**변경분만 전송**	Serial Notify, 재시작 뒤에도 serial 유지
보안	**전송 보안**	허용 IP · 연결 상한 + TCP-MD5 · TCP-AO · TLS · SSH



TCP 3323 (사실상 표준 포트)

시스템에는 HTTP API 로

기존 RP 의 형식을 따를 수 있는 곳은 따르고, 없는 기능은 새로 만들었습니다

Routinator 호환

- /api/v1/validity (경로 판정)
- /json-delta (바뀐 것만)
- /jsonext (VRP 의 출처 ROA, 옵션)

rpki-client 호환

- /json (VRP · ASPA · 라우터 키)
- /csv
- 같은 모양이라 비교 도구가 그대로 읽음

Haetae 고유

- AS · prefix 조회, 경로 판정
- ASPA 경로 검증 (AS_PATH 누출)
- 객체 · CA 탐색, unsafe VRP
- 만료 임박, 회차 이력, Prometheus
- /healthz (감시용 한 줄 판단)

출발지로 두 단계 : 관리망 (내부) 은 모든 경로, 인터넷 (공개) 은 조회 경로만 · 라우터 주소 같은 내부 정보는 공개 쪽에서 뺌

장애에 흔들리지 않게

입력은 인터넷의 믿을 수 없는 저장소입니다

1 입력을 믿지 않음

- 크기 · 개수 · 깊이 · 시간 상한
- 거대한 RRDP 요소는 메모리에 올리기 전에 차단
- https 만, 경로 탈출 차단, panic 은 그 CA 만

2 수집이 실패해도

- manifest 가 틀리면 직전 정상본으로 (RFC 9286)
- 망 오류면 직전 데이터, 죽은 저장소는 간격을 늘림
- 변경분은 모두 확인한 뒤 한 번에 적용

3 결과가 이상하면

- TA 하나라도 실패하면 반영 안 함 (첫 회차도)
- VRP 가 크게 줄면 경고 (보류 옵션)
- 거부된 CA 와 겹치는 VRP 를 unsafe 로 표시

root 가 아닌 전용 계정 · systemd 격리 · RTR / HTTP 연결 상한 · 전송 보안 (TCP-AO 등) 선택 · 파서 fuzz 시험

운영하기 쉽게

실행 파일 하나와 설정 파일 하나로 돌고, 멈추지 않고 설정을 바꿉니다

1 설정 파일 하나

- 모든 옵션을 설정 파일 키로
- 명령줄이 우선
- 모르는 키 · 틀린 값이면 시작 안 함

2 멈추지 않고 바꾸기

- systemctl reload 로 다시 읽기
- 새 값을 모두 확인한 뒤 한 번에
- 틀리면 지금 값 그대로 계속

3 TAL 내장

- 5 RIR 공식 TAL 을 실행 파일 안에
- 다른 패키지에 기대지 않음
- 시작 로그에 출처와 키 해시

4 접근 제어

- RTR · HTTP 각각 허용 목록
- HTTP 는 내부 / 공개 두 단계
- 허용 밖은 RTR 끊음 · HTTP 403

systemd 서비스 · 전용 계정 · 객체 크기 상한 20 MB · 시작 / 회차 / 경고가 모두 로그 한 줄

멈춰도 알아채고, 끊기지 않게

라우터에 주는 데이터가 끊기거나 오래되는 것을 가장 먼저 막습니다

1 판 올려도 안 끊김

- 같은 프로세스가 새 판으로 바뀜
- 라우터 연결 그대로, 약 2 초 멈춤
- 재시작해도 같은 session 으로

2 묵은 데이터 경보

- 1 시간 넘게 갱신이 없으면 경고
- 선택 : RTR 을 닫아 다른 RP 로
- 회차가 멈춰도 따로 확인

3 감시

- /healthz : 정상 200, 아니면 503
- 데이터 나이 · serial · RTR 상태
- Prometheus 지표

4 systemd

- watchdog : 멈추면 다시 띄움
- 종료 때 저장이 끝나길 기다림
- 죽은 저장소는 시도 간격을 늘림

매일 다른 RP (Routinator, rpki-client) 와 결과를 대조하고, 같은 입력이면 차이 0 을 확인합니다

v0.1 → v0.2.0 에서 바뀐 것

2026-11 · 전체 코드 검토에서 찾은 것을 고치고, 인터넷 시험 접속에 대비했습니다

보안

- 거대한 RRDP 요소 차단
- TA 인증서는 확인한 뒤에만 교체
- rsync · HTTP · RTR 크기 / 연결 상한
- SKI = 키 해시 검사 (RFC 6487, 경고)

안정

- manifest 실패 때 직전 정상본 (RFC 9286)
- 재시작 복원을 한 파일로 (어긋남 없음)
- 첫 회차 실패는 반영 안 함

운영

- /healthz · watchdog · 묵은 데이터 경보
- HTTP 내부 / 공개 두 단계
- 객체 크기 상한 4 → 20 MB
- 무중단 판 올리기 (RTR 안 끊김)

코드 검토의 고칠 것 10 건 모두 반영 · 파서 fuzz 시험 추가 · 같은 입력이면 rpki-client 와 차이 0 그대로

다음 단계

끝남 | 공개 | 지금 | 다음

기능

- RTR 전송 보안 (TCP-MD5 / TCP-AO / TLS / SSH)
- ASPA 경로 검증 · RSC
- 설정 파일 · reload · TAL 내장

v0.1 → v0.2.0

- ISC 라이선스, GitHub
- 코드 검토 반영 (보안 · 안정)

시험 운영

- 10 분 주기 상주, 매일 결과 대조
- 시험 라우터 RTR · 인터넷 시험 접속

병행 검증

- 기존 RP 와 몇 주 병행
- 라우터별 RTR 보안 · ASPA 지원 확인
- 상용망 적용 판단

Haetae

독립 구현 · 같은 결과 · 흔들리지 않는 운영

Go 표준 라이브러리만으로 만든 RPKI Relying Party · ISC 라이선스 오픈 소스

https://github.com/yunsog/haetae